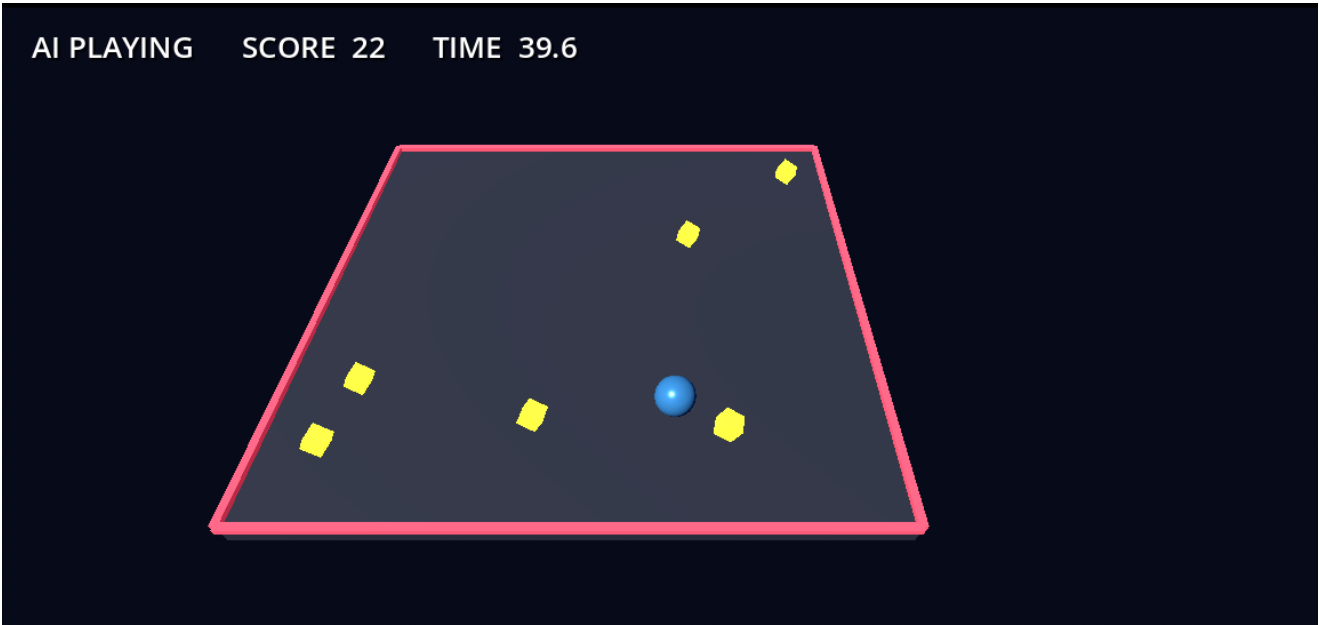


Ball Collector 3D

人間も遊べて、AI も強化学習で上達する 3D ゲーム

Godot 4.4 + godot-rl + Stable-Baselines3 / 2026 年 8 月 18 日



学習済み AI がプレイしている様子。青い球がプレイヤー、黄色がターゲット。

プレイヤー	取得数 / 60 秒	中身
学習前の AI (ランダム)	0.2	何も学習していない初期方策
手書きの貪欲方策	36	最寄りのターゲットへ直進するだけ
PPO で 18 万ステップ学習した AI	64	本レポートで作った方策

同じフィールド・同じルールを、人間はキーボードで、AI は強化学習で得た方策で遊ぶ。「学習させたら本当に上手くなるのか」を、数字で見えるところまで作った。

1 作ったもの

球体を転がして、60 秒でターゲットを何個集められるかを競う 3D ゲーム。球は **RigidBody3D** にトルクをかけて動かしているので、慣性で滑る。止まりたいときは進みたい方向と逆に入力して減速する、Marble Madness のような手触りになっている。

要素	内容
ルール	60 秒タイムアタック。ターゲットを取ると +1 点
ターゲット	常に 6 個。取ると別のランダム位置に再出現する
フィールド	30 × 30 の平らな板。縁を越えると場外落下 → 中央にリスポーン
操作	W A S D / 矢印キーで転がす、R でリスタート、Esc で終了
カメラ	球を追従。回転は固定（画面の奥 = ワールド -Z）
遊び方	ブラウザ（Web ビルド） / Windows ネイティブ / コンテナ内で直接実行

カメラの回転を固定した理由

カメラが回ると「画面の奥」と「ワールド座標の -Z」がずれる。人間は画面を見て操作し、AI はワールド座標で行動を出すので、回転を許すと両者にとっての「前」の意味が食い違ってしまう。カメラワークの自由度を捨てる代わりに、人間と AI がまったく同じ意味の操作をしている状態を保った。

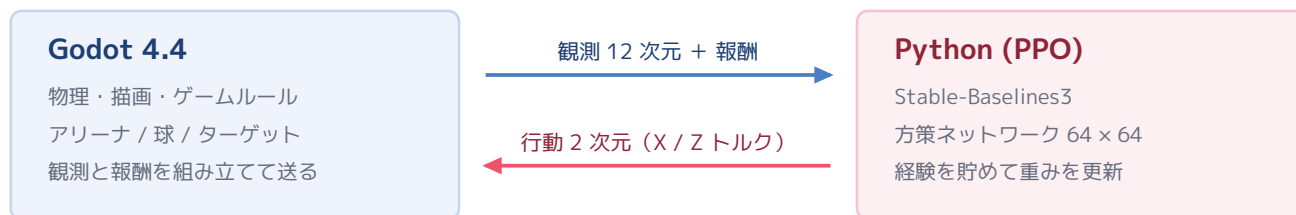
■ プロジェクトの構成

場所	役割
game/scripts/arena.gd	ルール・報酬・エピソード管理（1 プレイ分のフィールド）
game/scripts/player.gd	球の物理制御。人間入力と AI 行動の両方を受ける
game/scripts/ball_ai_controller.gd	観測・行動・報酬の定義（godot-rl の規約に沿う）
tools/train.py	PPO による学習
tools/play_ai.py	学習済みモデルにプレイさせて観る / スコアを測る
tools/godot_launcher.py	Godot プロセスの起動と学習環境の生成
models/ball_collector.zip	学習済みモデル

2 どんな学習をさせたか

使ったのは **強化学習**。正解のプレイデータを与えるのではなく、AI が自分で球を転がしてみ、点が入った動きの確率を上げていく方式にした。人間のプレイ記録は 1 件も使っていない。

アルゴリズムは **PPO** (Stable-Baselines3)。ネットワークは 64×64 の 2 層 MLP で、パラメータ数は数千しかない。画像は見せず、数値だけを渡している。この環境には GPU が無いので、CPU だけで学習しきれる規模に意図的に抑えた。



Godot がゲームを回し、Python が行動を決める。この往復そのものが学習になる (godot-rl が用意する TCP ソケットで、8 物理フレームごとに 1 往復)。

■ AI に見せているもの (観測 12 次元)

観測	次元	なぜ必要か
自分の速度 x, y, z	3	慣性で滑るので、今の勢いが分からないと止まらない
アリーナ内での自分の位置 x, y, z	3	縁に近いか = 落ちる危険があるかを判断する
最寄りターゲットへの相対位置 x, z	2	どちらへ向かうべきかの主信号
2 番目に近いターゲットへの相対位置 x, z	2	次の的も見えると経路取りが良くなる
最寄りターゲットまでの距離	1	近さそのものを明示的に渡す
残り時間	1	固定長エピソードを正しく扱うために必要 (下記)

残り時間を観測に入れた理由

エピソードは 60 秒で必ず終わる。もし残り時間を見せないと、AI から見た世界は「同じ状況なのに、あるときは続き、あるときは唐突に終わる」ものになってしまい、『この状態はこの先どれくらい得点につながるか』を見積もる価値関数が正しく学習できない。残り時間を観測に含めることで、いま見えている状態だけで先が決まる問題 (マルコフ決定過程) としてきちんと閉じる。

■ AI が出すもの (行動 2 次元)

ワールド X 方向と Z 方向のトルク指令、それぞれ $[-1, 1]$ の連続値。人間がキーを押したときに生成される入力とまったく同じ形式で、同じ `apply_torque()` に渡される。行動を決めるのは 8 物理フレームに 1 回 (action repeat = 8) なので、1 エピソード 3,600 フレーム中、AI の意思決定は 450 回。

■ 報酬の設計

事象	報酬	狙い
ターゲットを取った	+1.0	本来の目的そのもの
場外に落ちた	-1.0	縁に突っ込む動きを抑える
ターゲットに近づいた	近づいた距離 $\times 0.02$	学習の立ち上がりを速くする

距離シェーピングが効いた場面

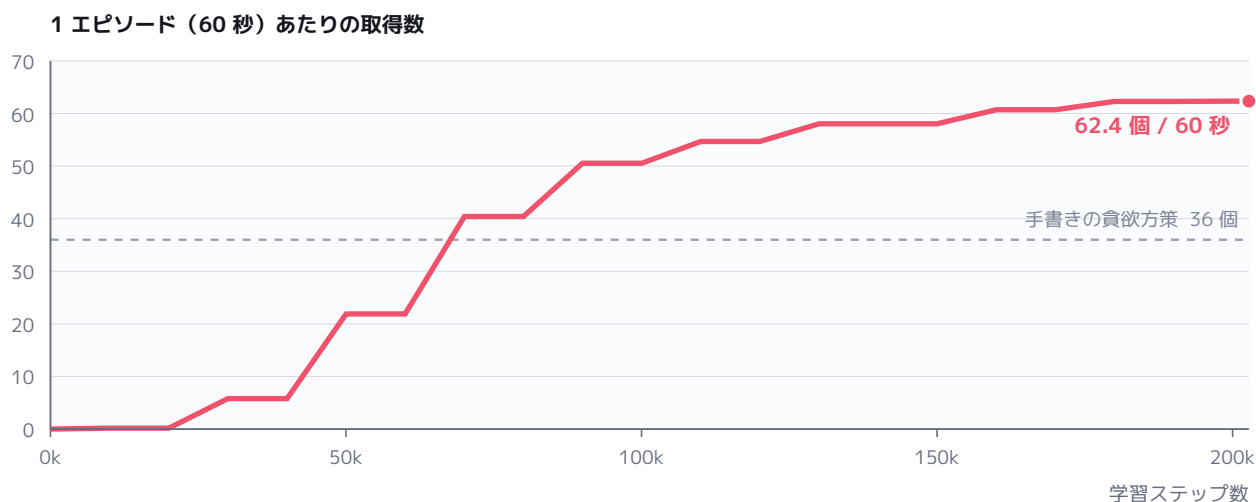
ターゲット取得の +1 だけだと、学習開始直後の AI は偶然ターゲットに触れるまで何の手がかりも得られない。ランダムに転がる球が 30×30 の板の上で的に当たる確率は低く、報酬がほぼ全ステップ 0 のまま時間だけが過ぎてしまう。「近づいた分だけ小さく加点」を足したことで、AI はまず『的の方を向いて進む』を覚え、そこから取得につながった。

■ 学習の設定値

項目	値	選んだ理由
方策ネットワーク	64×64 MLP	観測が 12 次元しかないので、これで十分足りる
学習率	$3e-4$	PPO の標準値。触る必要が無かった
割引率 γ	0.99	60 秒先まで見通せるように長めに取る
ロールアウト長 / バッチ	64 / 256	48 体ぶんを 1 回の更新にまとめる
エントロピー係数	0.001	探索を少しだけ残し、序盤で動きが固まるのを防ぐ
action repeat	8	60Hz の物理に対し、意思決定は 7.5Hz。学習を軽くする
同時実行エージェント数	48	16 アリーナ $\times 3$ プロセス
物理の倍速	40 倍	描画しないので、実時間より速く回せる

3 学習でできるようになったこと

実際に学習を回した結果が下のグラフ。横軸が学習ステップ数、縦軸が 1 エピソード（60 秒）あたりに取れたターゲット数。CPU のみで、最後まで約 1.2 分。



学習中の実測ログから作成。破線は比較用に書いた手書き貪欲方策の成績。

■ 何を覚えていったか

段階	取得数	見て分かる変化
学習前	0.2	その場でランダムに震えるだけ。落ちても気にしない
2 万ステップ	0.2	まだほぼ動けない。ここで距離シェーピングが効き始める
3 万ステップ	5.8	的の方向へ転がり出す。行き過ぎて通り過ぎることが多い
5 万ステップ	21.9	移動が滑らかになる。場外落下が目に見えて減る
9 万ステップ	50.5	慣性を見越して手前で減速するようになる
20 万ステップ	62.4	取った直後に次の的へ向き直る。移動が連続してムダがない

最終的に、最寄りの的へ直進するだけの手書き方策（36 個）を大きく上回った。差がついたのは主に 2 点。**慣性の扱い**（近づく前から逆向きのトルクをかけて減速し、行き過ぎない）と、**次の的まで見た経路取り**（2 番目に近い的を観測に入れてあるので、取った後の移動が短くなる向きからの的に入る）。どちらも明示的には教えていない。報酬を最大化した結果として出てきた振る舞いになっている。

※ グラフの 62.4 は学習中（方策からサンプリングして探索している状態）の平均。学習を止めて決定論的に動かすと 64 個になる。

学習を速くするためにやったこと

1 プロセスの中にアリーナを 16 面並べて 16 体を同時に走らせ、それを 3 プロセス、合計 48 体で経験を集めた。さらに物理シミュレーションを 40 倍速で回し、描画は不要なのでヘッドレスで実行している。この構成で環境側 6,779 steps/s、PPO の勾配計算を含めた実効で約 2,700 steps/s。20 万ステップが約 1.2 分で終わる。

4 学習をどうゲームに活かしているか

学習した方策を「別物」として横に置くのではなく、人間が遊ぶときとまったく同じコードを通すように作った。player.gd が入力元を切り替えるだけで、物理・ルール・エピソード長はすべて共通の経路を通る。



活かしている点	具体的にどうなっているか
スコアがそのまま比べられる	人間の 60 秒と AI の 60 秒が同じ長さであることを、AIController の reset_after (3,600 物理フレーム) を唯一の基準にすることで保証している。だから「AI は 64 個取れる」が、そのまま人間の目標値になる
AI のプレイを観られる	python tools/play_ai.py で、学習済みの方策が実際に遊ぶ様子をウィンドウで見られる。上達の中身（減速の仕方、的の回り方）を目で確認できる
ゲームバランスが学習で検証された	AI が 60 秒で 64 個取れる＝フィールドの広さ・的の数・トルクの強さが「詰まらずに動き続けられる」設定になっていることの裏付けになった。人間が遊んでも手持ち無沙汰にならない
報酬設計とルール設計が一致している	場外落下 -1.0 は、人間プレイでの「落ちると時間をロスする」感覚と同じ位置づけ。AI が落ちなくなった＝ルールの意図が素直に伝わる形になっていた

■ この構成にして良かったこと

ゲーム側と AI 側で仕様が二重管理になると、必ずどこかでずれる。エピソード長・トルクの強さ・ターゲット数といった「両方が知っている必要のある値」はゲーム側に 1 つだけ置き、AI はそれを観測として受け取る形にした。そのため、アリーナの広さやターゲット数を arena.gd で変えるだけで、人間のプレイにも AI の学習にも同時に反映される。ゲームバランスを触るたびに学習コードを直す、という手戻りが起きない。

5 つまずいた点と、どう回避したか

問題	対処
Godot 単体で学習済みモデルを動かせない	godot-rl の ONNX 推論部は C# 実装で、この環境の Godot は非 .NET ビルドだった。そこで「Python が推論し、Godot が描画する」構成にした。将来 .NET ビルドへ移れるよう models/ball_collector.onnx は出力してある
学習ログに「何個取れたか」が出せない	godot-rl 0.8.1 の GodotEnv.step_recv() が、Godot から届いた info を読み捨てていた。受け取り側を差し替えて info を通し、報酬ではなく取得数で進捗を見られるようにした。報酬は距離シェーピングを含むので直感的に読めないため、これは重要だった
複数プロセスで学習を回せない	本家のラッパーが「実行ファイルが無いなら複数プロセス不可」と決め打ちしていた。自前で Godot プロセスを起動して接続する形にして、48 体並列を実現した
コンテナ内のプレイが 25fps しか出ない	描画自体（CPU ソフトウェア描画）は 65fps 出しており、遅いのは X11 のフレーム転送だと実測で切り分けた。解像度を下げても頭打ちになる。そこで Web ビルドと Windows ネイティブビルドを用意し、そちらでのプレイを推奨する形にした

■ 実測した性能

項目	実測値
学習スループット（16 アリーナ × 3 プロセス、40 倍速）	6,779 steps/s
PPO の更新を含めた実効速度	約 2,700 steps/s
20 万ステップの学習時間（CPU のみ）	約 1.2 分
メモリ（Godot ヘッドレス 1 プロセス / Python 側）	85 MB / 672 MB
描画 FPS（ローカル、1152 × 648）	65.3 fps
描画 FPS（X11 転送、1152 × 648）	12.0 fps

■ 次にやるとしたら

障害物と周辺認識 — 今のフィールドは平坦で、AI は的の位置しか見ていない。プラグイン同梱の RaycastSensor3D を観測に足せば、壁を避ける行動まで学習できる。

ゲートくぐり — 空中のリングを連続でくぐる 3D タスク。観測・行動・報酬の骨組みはそのまま流用できる。

サバイバルモード — 的を取ると残り時間が回復するルール。arena.gd の変更だけで済む。